

#6 Optimization for Biney's Procedural Compression

Biney

July 6, 2026

Abstract

This paper proposes an extension to procedural compression by allowing multiple related files to share common procedural models instead of independently storing separate procedural algorithms for every file. Rather than optimizing each file in isolation, the compression problem is reformulated as a joint optimization over an entire archive, where one or more shared procedural algorithms are amortized across many files while each file stores only its corresponding seed, parameters, metadata, or correction information. This formulation reduces redundant storage of procedural models and introduces a global optimization objective that minimizes the total archive size instead of individual file sizes. The paper presents the mathematical formulation of this objective and discusses practical extensions incorporating computational constraints such as search time, reconstruction time, and compute budget. This framework provides a foundation for future AI-guided and heuristic search methods capable of discovering shared procedural representations for large collections of related data.

Shared Procedural Models Across Multiple Files

Instead of searching for a unique procedural program (algorithm + seed) for every individual file, the compressor could search for a shared procedural algorithm across N related files, where each file stores only its own reconstruction representation (parameters, metadata, residual corrections, etc.).

Instead of storing

$$\begin{aligned} \text{File}_1 &\rightarrow \text{Algorithm}_1 + \text{Seed}_1 \\ \text{File}_2 &\rightarrow \text{Algorithm}_2 + \text{Seed}_2 \\ &\vdots \\ \text{File}_N &\rightarrow \text{Algorithm}_M + \text{Seed}_N \end{aligned}$$

the archive could instead store

$$\begin{aligned} &\text{Algorithm}_1 \\ &\vdash R_1 \\ &\vdash R_2 \\ &\vdash R_3 \\ &\vdash R_N \end{aligned}$$

The cost of storing one procedural algorithm is therefore amortized across many files, potentially reducing the average compressed size per file. This changes the optimization from compressing individual files to compressing an entire archive.

Before

For each file, the compressor independently searches for

$$\min_{A_i, S_i} (|A_i| + |S_i|)$$

requiring a separate search over the procedural program space for every file.
The total archive size is

$$\sum_{i=1}^N (|A_i| + |S_i|).$$

After

Instead, the compressor searches for a smaller collection of shared algorithms together with a complete per-file reconstruction representation for every file.

The optimization objective becomes

$$\min_{\{A_j\}, \{R_i\}, \{Y_i\}} \left(\sum_{j=1}^M |A_j| + \sum_{i=1}^N |R_i| + |G| \right)$$

where

- N = total number of files.
- M = total number of shared procedural algorithms, where typically $M \ll N$.
- A_j = the j -th shared procedural algorithm.
- R_i = the complete per-file reconstruction representation required to reconstruct file i .
- Y_i = the identifier of the shared procedural algorithm assigned to file i .
- G = archive-wide information stored once for the entire archive (headers, indexes, version information, checksums, global metadata, etc.).

Since the archive-wide information is stored only once regardless of the number of files, it contributes a single global term G rather than one term per file.

The per-file reconstruction representation may be written explicitly as

$$R_i = (Y_i, P_i, D_i),$$

where

- Y_i = identifier of the shared procedural algorithm assigned to file i .
- P_i = procedural parameters (seed) required by that algorithm.
- D_i = residual or correction data required for exact lossless reconstruction.

Accordingly,

$$|R_i| = |Y_i| + |P_i| + |D_i|.$$

For every file, the decoder is required to satisfy

$$\boxed{\text{Decode}(A_{Y_i}, R_i) = F_i}$$

where

- A_{Y_i} = the shared procedural algorithm assigned to file i .
- R_i = the complete per-file reconstruction representation stored for file i .
- F_i = the original file.

This reconstruction constraint ensures that every stored representation reproduces its corresponding original file exactly, thereby guaranteeing lossless compression.

Thus, the optimization objective may also be written explicitly as

$$\boxed{\min_{\{A_j\}, \{R_i\}, \{Y_i\}} \left(\sum_{j=1}^M |A_j| + \sum_{i=1}^N (|Y_i| + |P_i| + |D_i|) + |G| \right)}$$

This objective represents the arrangement that minimizes the total archive size by jointly optimizing

- how many shared algorithms should exist,
- which files should share each algorithm,
- when creating a new algorithm is preferable to reusing an existing one,
- and the trade-off between the size of shared algorithms and the sizes of their corresponding per-file reconstruction representations.

Rather than independently minimizing every file, the compressor minimizes the storage cost of the archive as a whole while satisfying the reconstruction constraint

$$\text{Decode}(A_{Y_i}, R_i) = F_i \quad \forall i \in \{1, \dots, N\}.$$

Can this objective be improved?

Yes. The above is the simplest formulation. A more realistic objective would also include practical constraints such as search time and reconstruction time.

For example,

$$\begin{aligned}
 &\text{Minimize} && \sum_{j=1}^M |A_j| + \sum_{i=1}^N |R_i| + |G| \\
 &\text{subject to} && \text{Decode}(A_{Y_i}, R_i) = F_i, \quad \forall i, \\
 & && \text{Search Time} \leq T, \\
 & && \text{Reconstruction Time} \leq R, \\
 & && \text{Compute Budget} \leq C, \\
 & && \text{Size of Compressed Archive} \leq K.
 \end{aligned}$$

Penalty-Based Optimization

An equivalent formulation incorporates the constraints directly into the optimization objective using penalty terms. Rather than immediately rejecting solutions that violate a constraint, the optimizer assigns them a higher objective value, naturally guiding the search toward smaller archive representations that also satisfy the desired computational limits.

The resulting objective becomes

$$\begin{aligned}
 \min \quad & \sum_{j=1}^M |A_j| + \sum_{i=1}^N |R_i| + |G| \\
 & + \lambda_D L_{\text{decode}} + \lambda_T L_{\text{search}} + \lambda_R L_{\text{reconstruction}} + \lambda_C L_{\text{compute}} + \lambda_K L_{\text{archive}}
 \end{aligned}$$

where

- L_{decode} penalizes incorrect reconstruction.
- L_{search} penalizes search times exceeding T .
- $L_{\text{reconstruction}}$ penalizes reconstruction times exceeding R .
- L_{compute} penalizes compute usage exceeding C .
- L_{archive} penalizes compressed archive sizes exceeding K .
- $\lambda_D, \lambda_T, \lambda_R, \lambda_C$, and λ_K are weighting coefficients that determine the importance of each penalty.

This formulation is particularly useful for AI-guided, heuristic, and other gradient-free optimization methods, as it converts multiple constraints into a single objective that can be directly optimized while encouraging solutions that remain computationally practical.

This optimization therefore asks the compressor to find the smallest archive representation that

- reconstructs every original file exactly,
- satisfies the computational constraints,
- and minimizes the total archive storage.

This tells the compressor:

“Find the smallest possible archive representation while remaining computationally practical.”

The search itself could then be guided by AI, SyGuS, heuristic search, clustering, reinforcement learning, or other combinatorial optimization techniques to approximate this objective without exhaustively exploring the entire search space.

Additional Note on Search Time

Although the theoretical search space remains enormous, the practical search is bounded by the configurable search time T . Increasing T allows the compressor to explore a larger portion of the search space, increasing computational cost but also the likelihood of discovering increasingly smaller procedural representations.

For example, even if there are theoretically on the order of 2^{4000} possible procedural programs of length 500 bytes (assuming an unrestricted binary representation), setting $T = 76$ hours simply means the compressor searches for at most 76 hours before halting. Rather than attempting an exhaustive search of the entire space, it explores only the subset of candidate programs reachable within that time budget.